

**SIMATS ENGINEERING
ASSESSMENT TOOLS**

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 40%

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

CLASS TEST

Code of Conduct:

I V. Smylika(192372115) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Smylika V.
Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design a PDA to accept strings with 'n' occurrences of 'a' followed by 'n' occurrences of 'b' where 'n' is a non-negative integer.
i) $L = \{a^n b^n\}$

2. Convert the given context-free grammar (CFG) into a Pushdown Automaton (PDA) that recognizes the same language.

$$E \rightarrow E+E \mid E^*E \mid (E) \mid I$$
$$I \rightarrow Ia \mid Ib \mid I0 \mid I1 \mid a \mid b$$

3. Design a TM to accept the strings with 'n' occurrences of 'a' followed by 'n' occurrences of 'b' where 'n' is a non-negative integer.
i) $L = \{0^n 1^n \mid n \geq 1\}$

4. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

5. Construct a Turing Machine to recognize the language. Write the logic used for the design :
the formal definition

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

class Test - 02.
CO4 Assessment Tool 3

V. Smijika
192312115
CSA1301

or b. $L = \{a^n b^n\}$

PDA:

- The PDA uses its stack to count the number of as.
- For every A, push one A onto the stack.
 - When Bs begin, pop one A every for every B.
 - Accept when all A's have been popped and the input is accepted.
 - $n=0$ gives the empty string ϵ , so ϵ is also called accepted.

Let $M = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$

- where
- $Q = \{q_0, q_1, q_f\}$
 - $\Sigma = \{a, b\}$
 - $\Gamma = A, z_0$
 - Initial state $= q_0$
 - Initial stack symbol $= z_0$
 - $F = q_f$

Transitions

For reading a and pushing:

$$\delta(q_0, a, z_0) = \{(q_0, A z_0)\}$$

$$\delta(q_0, a, A) = \{(q_0, AA)\}$$

When b starts, pop one A:

$$\delta(q_0, b, A) = \{(q_1, \epsilon)\}$$

Continue popping

$$\delta(q_0, b, A) = \{(q_1, \epsilon)\}$$

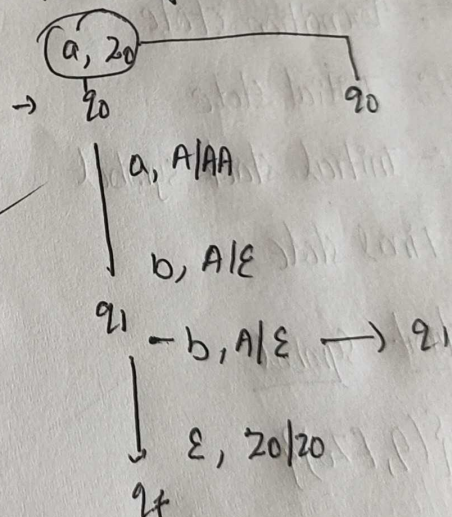
When all a's are removed:

$$\delta(q_1, \epsilon, z_0) = \{(q_f, z_0)\}$$

For $n=0$:

$$\delta(q_0, \epsilon, z_0) = \{(q_f, z_0)\}$$

State diagram:



2. Convert CFG to PDA

Given:

$$E \rightarrow E + E \mid E * E \mid (E) \mid \epsilon$$

$$I \rightarrow I_0 \mid I_1 \mid I_2 \mid I_3 \mid I_4 \mid I_5 \mid I_6 \mid I_7 \mid I_8 \mid I_9$$

We construct a PDA using the standard CFG-to-PDA conversion.

Logic:-

1. Starts with the start variable E on the stack.
2. If the top of stack is a variable, replace it using a grammar production.
3. If the top is terminal, match it with the input symbol and pop it.
4. Accept when the entire input and stack are consumed.

PDA definition

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$$

where $Q = \{q_0, q, q_f\}$

$$\Sigma = \{a, b, 0, 1, +, *, (,)\}$$

Γ = Transition state

q_0 :- Initial state

Z_0 :- Initial stack symbol

q_f = Final state

Step 1:- Push start symbol

$$\delta(q_0, \epsilon, Z_0) = \{(q, EZ_0)\}$$

Step 2:- Replace variables according to productions

For E : $(q, E+E)$

$$\delta(q, E, E) = \{(q, (E))\}$$

$$(q, I)$$

Step 3:- Match terminals

For every terminal a :

$$\delta(q, a, a) = \{(q, \epsilon)\}$$

where

$$a \in \{a, b, 0, 1, +, *, (,)\}$$

Step 4:- Accept
Only Z_0 remains

$$\delta(q, \epsilon, Z_0) = \{(q_f, Z_0)\}$$

Thus, the PDA recognizes exactly the language generated by

even :- $L = \{0^n 1^n \mid n \geq 1\}$

Logic : The TM repeatedly matches one 0 with one 1.

Use markers : $x =$ processed 0

$y =$ processed 1

Definition :

$M = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$

where

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, x, y, B\}$

$q_0 =$ starting state

$B =$ Blank

$F =$ Final state

$\delta(q_1, x) = (q_1, x, L)$

$\delta(q_2, y) = (q_2, y, L)$

$\delta(q_2, B) = (q_0, B, R)$

Accept when everything is matched

$\delta(q_0, y) = (q_0, y, R)$

$\delta(q_0, B) = (q_4, B, R)$

$\therefore$ The Accepted string

Transition :

$\delta(q_0, 0) = (q_1, x, R)$

Skip processed zeros :

$\delta(q_0, x) = (q_0, x, R)$

Find matching 1 :

$\delta(q_1, 0) = (q_2, 0, R)$

$\delta(q_1, y) = (q_1, y, R)$

$\delta(q_1, 1) = (q_2, y, L)$

Return left :

$\delta(q_2, 0) = (q_2, 0, L)$

01

0011

000111

00001111

..

4. Given: $L = \{a^n b^m \mid n \geq 1\}$

Here the no of b's must be twice the number of a's

Logic:-

For every a, push two stack symbols.

Then, for every b, pop one stack symbol.

Ex:- aaabbbbb

There are 3 a's and 6 b's

Stack after reading aa:

AAAAAA

Each b removes one A.

PDA definition:

$M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

where $Q = \{q_0, q_1, q_f\}$

$\Sigma = \{a, b\}$

$\Gamma = \text{Transition state } \{A, Z_0\}$

$F = \text{Final state } \{q_f\}$

Transitions:-

For each a, push two A's:

$\delta(q_0, a, Z_0) = \{(q_0, AA Z_0)\}$

$\delta(q_0, a, A) = \{(q_0, AAA)\}$

Next transition adds two more A's because one existing A is replaced by three A's

$\delta(q_1, b, A) = \{(q_1, \epsilon)\}$

when stack reaches bottom

$\delta(q_1, b, A) = \{(q_1, \epsilon)\}$

Ex:- aaabbbb

After aa:

AAAA

A $\therefore$ AAAAbb

Then

b $\rightarrow$ AAA

b $\rightarrow$ AA

b $\rightarrow$ A

b $\rightarrow$ Z₀

$\therefore$ it is accepted

Ques: $L = \{a^n b^n c^n \mid n \geq 1\}$

This language requires equal numbers of a, b, and c.
A PDA cannot generally recognize the language, but a Turing Machine can.

Logic:

- x for processed a
- y for processed b
- z for processed c

Definition:

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{a, b, c\}$$

Γ = Transition state

$$\{a, b, c, x, y, z, B\}$$

q_0 = Initial state

F = Final state

Transition

$$\delta(q_0, x) = (q_0, x, R)$$

$$\delta(q_0, a) = (q_1, x, R)$$

Find corresponding

$$\delta(q_1, a) = (q_1, a, R)$$

$$\delta(q_1, y) = (q_1, y, R)$$

$$\delta(q_1, b) = (q_2, y, R)$$

Find corresponding c:

$$\delta(q_2, b) = (q_2, b, R)$$

$$\delta(q_2, y) = (q_2, y, R)$$

when c is found.

$$\delta(q_2, c) = (q_3, z, L)$$

Return to beginning

In q_3 , move left

when blank is reached

$$\delta(q_3, B) = (q_0, B, R)$$

Final checking

when no unmarked a remains the tape.

If only x, y, z

remain, accept.

$$\delta(q_0, B) = (q_f, B, R)$$

If an unmatched b
reject.

Ex:-

aaa'bbbccc

TH:-

aaa'bbbccc

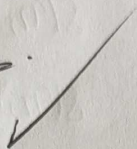
xaa'ybb'zcc

xxa'y'bb'z'cc

xxx'y'y'zzz

Since each a, b and c has been matched exactly once.

∴ Accepted



SIMATS ENGINEERING ASSESSMENT TOOLS

FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100